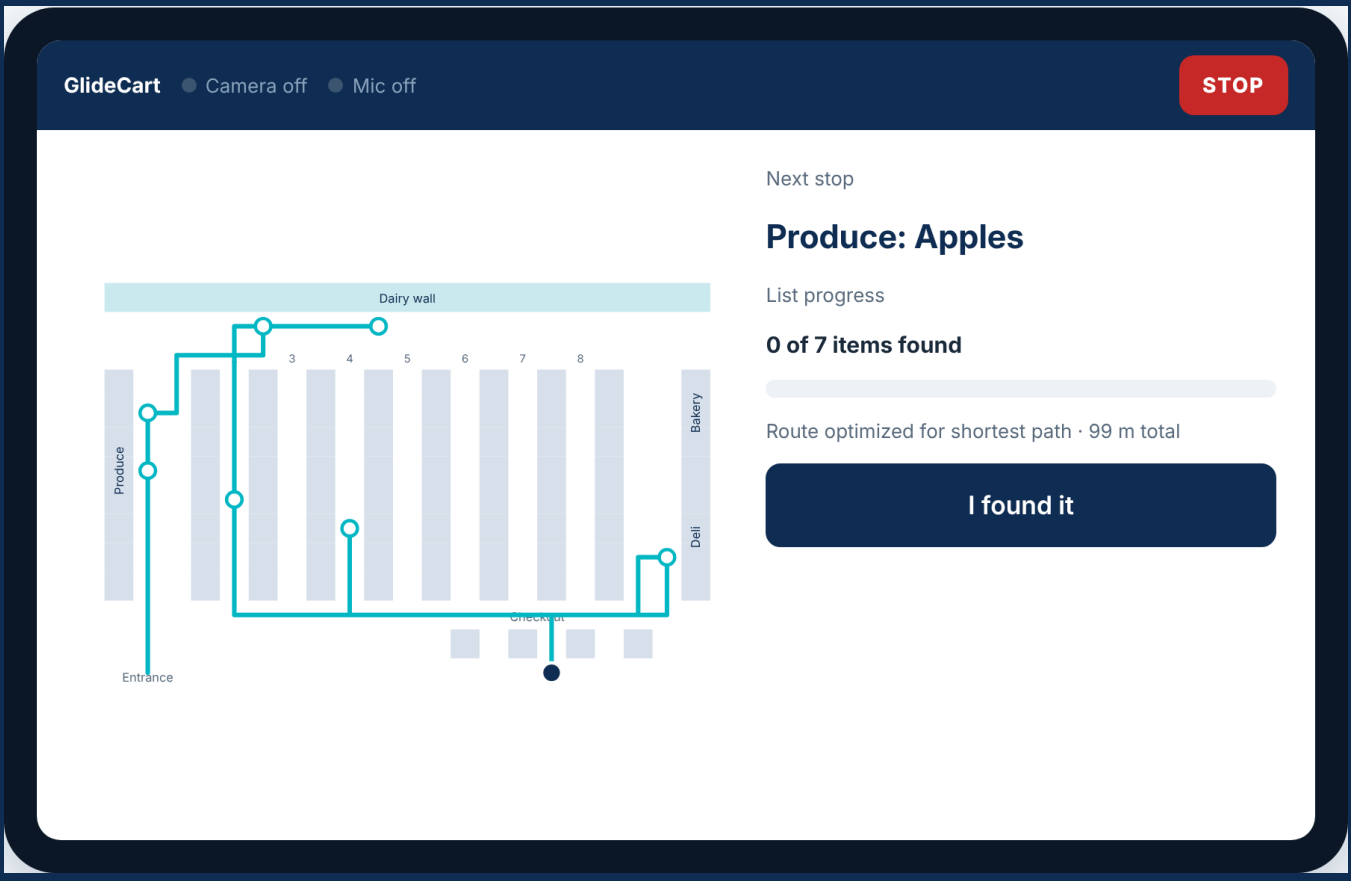
PORTFOLIO CASE STUDY · LOGICAL CODERS

GlideCart

A smart grocery cart that plans the shortest route, listens on request, and keeps shopper data to itself.

Product profile · AI assistant and voice design · Machine learning automation · Test report



Anél Henning UX/UI designer and developer · Tampa, FL · October 2026

Concept prototype. Fictional store, products and prices.

01 · PRODUCT PROFILE

GlideCart at a glance

GlideCart is a concept for a self-guiding grocery cart. The shopper picks a list, the cart works out the shortest walk through the store, and a screen on the handle leads the way. It was designed around one rule: be useful without watching people.

Field	Detail
Product	GlideCart smart cart: web prototype of the cart screen plus a store-planning demo
Status	Concept prototype. Fictional store (Bayside Fresh Market), products and prices. Not a commercial product.
Who it serves	Shoppers who want a faster trip, shoppers with limited mobility, and store order pickers; store managers who run a pilot
Built with	One self-contained HTML file: HTML, CSS and plain JavaScript. SVG store maps, Web Crypto (SHA-256) for the price-file check. No framework, no server.
Live build	anelhenning2-collab.github.io/portfolio/apps/glidecart (also published as a Lovable build)
Pages	Home · How it works (route race) · Security · For stores (pilot plan) · Cart screen (interactive tablet)
My role	Concept, research, UX and UI design, front-end build, routing algorithm, security review and testing

54%
shorter walk on the demo list (216 m to 99 m)

12 / 12
automated tests passed on the shipped code

97%
of small lists routed perfectly, checked by brute force

2.7 ms
to plan a full 16-item route

The problem

In a typical store a shopper with a mixed list zig-zags or walks every aisle to be sure nothing is missed. That costs time for shoppers and money for stores whose staff pick online orders. Earlier smart-cart ideas leaned on cameras that watch people and on cloud processing, which shoppers distrust. GlideCart asks how much help a cart can give while keeping cameras pointed at products and audio on the cart.

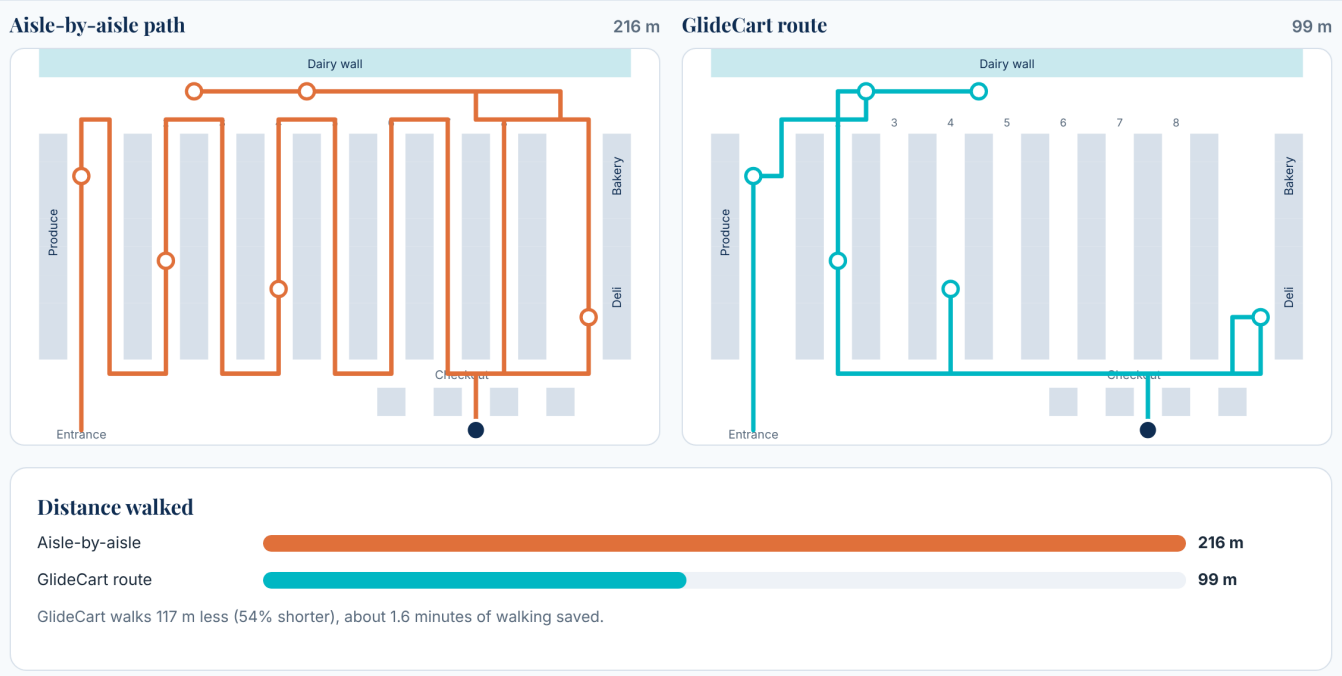
About the designer

Anél Henning runs Logical Coders in Tampa, FL. She brings fifteen years of multi-channel design for corporate, healthcare and non-profit clients, a BFA in Graphic Design from MICA, a BS in Computer Science, and an MS in Cybersecurity Management from Purdue Global (expected November 2026). Retail operations experience at Publix shaped the store problems GlideCart tries to solve.

02 · WHAT IS BUILT TODAY

The working prototype

Everything below runs in the browser today. Screens were captured from the live code for this document.

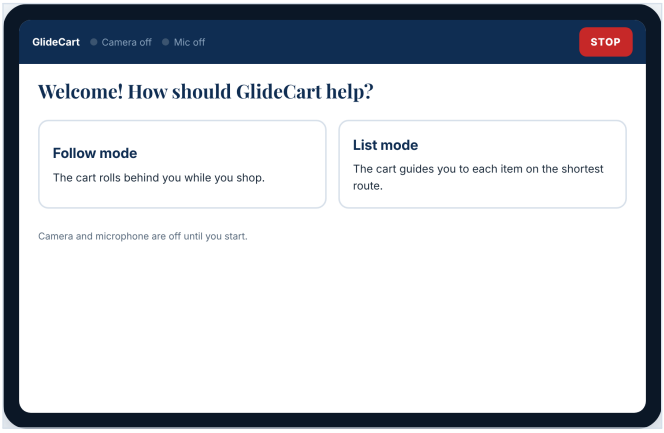


How it works page: the same six-item list walked aisle by aisle (orange, 216 m) and by GlideCart (teal, 99 m).

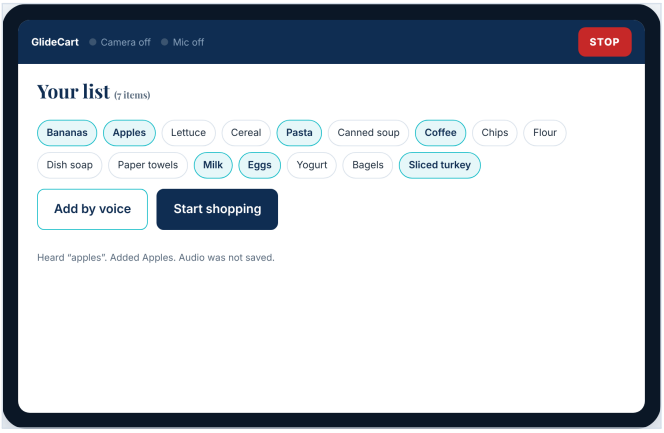
Features in the code

Feature	What it does	How it works
Route race	Shopper picks any of 16 items; two carts animate through the store	Store is a 23 × 14 grid of 1.5 m cells. Breadth-first search finds shortest paths between cells.
Shortest route	Orders the list to minimise walking	Nearest-neighbour order, then 2-opt swaps until no swap shortens the trip. Walking speed 1.2 m/s for time saved.
Price-file check	Blocks a tampered supplier price file	Recomputes the file's SHA-256 digest and compares it with the published one. A one-cent edit flips the verdict to BLOCKED.
Cart screen	Tablet UI: Follow or List mode, voice add, step-by-step route, checkout	Simulated sensors. Camera and mic lights show their state at all times. A red STOP button overlays every screen.
Privacy controls	Delete my trip data at checkout	Clears trip data on the cart; voice demo confirms audio was not saved.
Accessibility	Keyboard, screen reader and motion settings	Focus outlines, ARIA labels on maps and buttons, live regions for results, reduced-motion support.

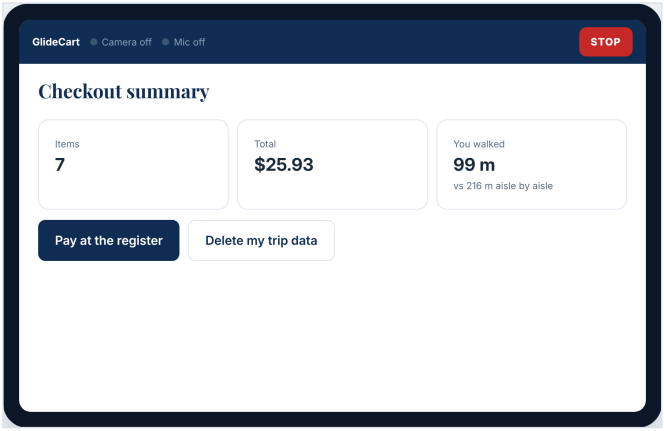
The cart screen, step by step



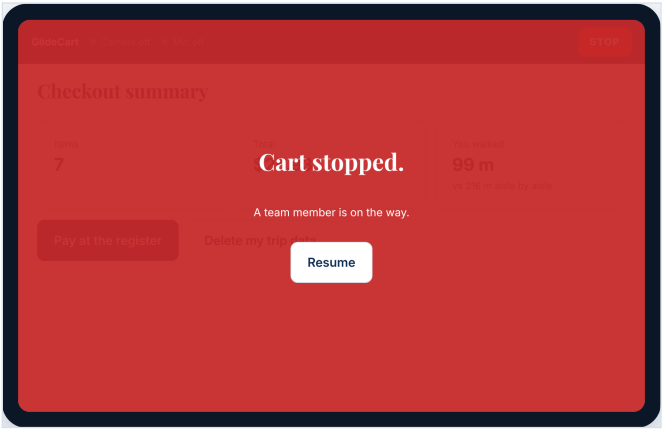
Welcome: pick Follow mode or List mode. Sensors stay off until you start.



Voice add: the mic light turns on, "apples" is added, and the screen confirms audio was not saved.



Checkout: items, total from the verified price file, distance, and a delete-my-data button.



STOP: one tap halts the cart and calls a team member.

Signed price files

Every supplier price file must match the digest the supplier signed. If one character changes, the file is blocked.

☑ Send tampered file

```
supplier=Gulf Coast Produce Co (fictiona
date=2026-10-28
sku,item,price
4011,Bananas,0.69
4131,Apples,0.49
4640,Lettuce,1.99
```

BLOCKED · signature does not match

Signed digest 5ebf636355c23f4d… · received 1773a9ba520dc5c7…

Private sensors

- A light turns on whenever the camera or microphone is in use.
- The camera looks at products, not people; no images of people are stored.
- Voice commands are processed on the cart, not sent to the cloud.
- Trip data can be deleted from the checkout screen.

Post-quantum encryption

Payment and sync traffic use NIST's post-quantum key exchange, ML-KEM. In the Logical Coders tests it added about 262 microseconds per connection.

The 2025 concept proposed quantum key distribution; the 2026 audit replaced it with this standard because it runs on ordinary hardware.

Security page with "Send tampered file" ticked: Apples changed from 2.49 to 0.49, so the digest no longer matches and the file is blocked.

03 · AI ASSISTANT DESIGN

Glide: a shopping assistant that stays on the cart

The next version adds a conversational assistant to the handle screen. It answers by text or voice, acts on the shopper's list, and only speaks from facts it can check: the store map and the verified price file.

Design principles

- **Grounded answers only.** Prices come from the signed price file and locations from the store map. If an item is not in the catalog the assistant says so instead of guessing.
- **On the cart, not in the cloud.** Speech recognition and intent detection run on the cart's own processor. Audio is discarded once it is turned into text.
- **Visible sensors.** The mic light is on whenever the assistant is listening, matching the promise already on the prototype.
- **Touch always works.** Every action the assistant can take is also one tap away, for shoppers who cannot or would rather not speak.
- **Safety first.** The physical STOP button never depends on the assistant.

Sample conversation

You

Add oat milk, and where is the coffee?

Assistant

This store's list does not have oat milk. The closest match is **Milk** on the Dairy wall. Add it? **Coffee** is in Aisle 4. I have put it on your route.

You

Yes, add milk. How much are the bananas?

Assistant

Added Milk and re-planned your route. Bananas are **$0.69**, from today's verified supplier price file.

You

Stop.

Assistant

Cart stopped. A team member is on the way. Tap Resume when you are ready.

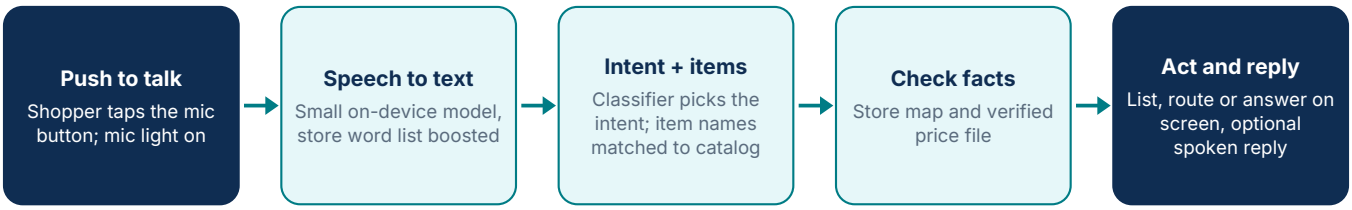
What the assistant understands

Intent	Example the shopper might say	What the cart does
add_item / remove_item	"Add yogurt and bagels." "Take off the chips."	Matches each name to the catalog, updates the list, re-plans the route
find_item	"Where is the coffee?"	Answers "Aisle 4, stop 5 on your route" and highlights it on the map
price_check	"How much are the bananas?"	Reads $0.69 from the verified price file and says where the price came from
substitute	"Add oat milk."	Not in catalog: offers the closest match (Milk, Dairy wall) and asks before adding
navigate	"Next item." "I found it." "Undo."	Moves the route forward or back, same as the on-screen buttons
mode	"Follow me." "Switch to list mode."	Changes mode; camera light turns on in Follow mode
read_list / summary	"What is left?" "How far have I walked?"	Reads remaining stops; shows distance and total
help / stop	"I need help." "Stop."	Calls a team member; voice stop is an extra path, never the only one
privacy	"Delete my data."	Confirms, then clears trip data, same as the checkout button

04 · VOICE COMMANDS

Push to talk, processed on the cart

Voice turns the list screen hands-free for shoppers carrying a basket, a child or a cane. The design keeps the prototype's promise: voice commands stay on the cart.



Audio is held in memory only for the length of the command, then discarded. Only the text command is kept, and only until the trip ends.

Command reference

Say	Result	Works in
"Add [item]" · "Add [item] and [item]"	Adds one or more items; route re-planned	List mode
"Remove [item]"	Takes the item off the list	List mode
"Where is [item]?"	Aisle and stop number, shown on the map	Any screen
"How much is [item]?"	Price from the verified file	Any screen
"Next" · "I found it" · "Undo"	Moves through the route	Route screen
"Follow me" · "List mode"	Switches mode	Any screen
"What is left?"	Reads remaining items aloud	Route screen
"Checkout"	Opens the checkout summary	Route screen
"Stop" · "Help"	Halts the cart and calls staff	Any screen, plus Follow mode listening
"Delete my data"	Clears trip data after a confirm	Checkout

Hard design questions and the answers chosen

Question	Decision
Always listening or push to talk?	Push to talk. An always-on microphone in a public store breaks the privacy promise. The one exception is Follow mode while the cart is moving: a tiny keyword model listens only for "stop", the mic light stays on, and nothing is recorded.
What if the cart mishears?	Changes to the list show on screen with Undo for five seconds. Low-confidence matches ask "Did you mean Milk?" instead of acting.
Store noise and accents	The recognizer is tuned on store vocabulary (the 16 catalog names and commands) and tested across accents and 70 dB background noise before launch (see test AI-01 and AI-08).

Question	Decision
Shoppers who do not speak	Every voice command has a button. Voice is a shortcut, never the only way.

05 · MACHINE LEARNING AUTOMATION

Five jobs the cart learns to do

Each model below automates a step a person does by hand today. Every one has a plain fallback, runs where the data lives, and has a number it must hit before it ships.

Task automated	Algorithm	Learns from	Replaces	Must reach
1. Confirm "I found it" when an item goes in the basket	Image classifier (MobileNetV3-Small, fine-tuned), on the cart	Labelled photos of the store's products, taken in the basket	Tapping "I found it"	Top-1 accuracy 97%+, wrong confirmations under 1%
2. Suggest list items ("You usually add eggs with milk")	Association rules (FP-Growth)	Past baskets, only for shoppers who opt in	Remembering the list	Precision of top 3 suggestions 30%+; easy to turn off
3. Predict trip time and avoid crowds	Random Forest regressor for trip time; gradient boosting for aisle crowding by hour	Route features and cart traffic logs (no personal data)	Fixed 1.2 m/s estimate; congestion-blind routing	Trip-time error within 15%; fewer stops in crowded aisles
4. Flag suspicious price changes even when the file is validly signed	Isolation Forest anomaly detection	Each item's price history	A person spotting a bad price after shoppers are charged	Catches 95%+ of injected errors, false alarms 2% or less
5. Understand voice commands	Text classifier (TF-IDF + logistic regression baseline, small transformer upgrade) plus fuzzy catalog matching	Labelled command phrases	Fixed keyword matching	Intent accuracy 95%+ on 500 test phrases

Starting point: models already prototyped

The GlideCart research labs (github.com/AnelHenning2-collab, glidecart-labs) already train two of these ideas in Python with scikit-learn. They use synthetic data, so the numbers show the method works, not how it would do in a real store.

Lab model	Task	Result on held-out synthetic data	What it tells us
Random Forest regressor (Lab 1)	Predict route completion time from 8 order features	R^2 0.754, mean error 20.9 s (Gradient Boosting: R^2 0.733)	Usable for trip-time estimates. "Has deli stop" and number of stops matter most, so deli queues belong in the model.
Random Forest regressor (Lab 2)	Predict encryption latency per payment	R^2 0.955, mean error 0.21 ms	Post-quantum encryption adds very little delay for small retail payloads.

Lab model	Task	Result on held-out synthetic data	What it tells us
Gradient Boosting classifier (Lab 2)	Recommend an encryption algorithm	100% accuracy	Too perfect: the labels come from a fixed rule the model can copy. Needs real or noisier data before the score means anything (finding F4).

How the models stay trustworthy

- **Human in the loop.** Flagged prices go to a store manager; suggested items need a tap to add.
- **Fallbacks.** If the camera model is unsure, the shopper taps as today. If the trip-time model is offline, the fixed estimate returns.
- **Data minimisation.** Product photos are cropped to the basket; no images of people are kept. Basket history is opt-in and deletable.
- **Monitoring.** Accuracy is re-checked weekly on fresh labelled samples; a model that drops below its threshold is switched off automatically.

06 · TESTING

What was tested, and what was found

Testing has two parts: automated tests run on the shipped prototype for this case study, and a test plan the AI features must pass before launch.

Approach

The routing model was copied line for line from the live page into a Node.js test harness, so the tests check the real code rather than a rewrite. Random shopping lists use a fixed seed so every run is repeatable. For small lists the route was compared with the true best route, found by trying every possible order.

ID	Test	Result	Evidence
GC-01	Every item on the list appears exactly once in the route	**PASS**	300 random lists, sizes 2-16
GC-02	Route never steps through a shelf and moves one cell at a time	**PASS**	200 routes checked cell by cell
GC-03	Route starts at the entrance and ends at checkout	**PASS**	100 routes
GC-04	Aisle-by-aisle baseline is a connected path through every aisle	**PASS**	216 m baseline, passes within 1 cell of all 16 items
GC-05	2-opt never makes the route longer than nearest-neighbour alone	**PASS**	500 lists; 2-opt improved 300 (60%), tied the rest
GC-06	GlideCart route is never longer than the aisle-by-aisle path	**PASS**	1,000 lists; savings 19-81%, mean 47.8%
GC-07	Route is close to the true optimum (brute force, lists of 3-7)	**PASS**	150 lists; optimal in 146 (97%); worst gap 6.9%; NN-only worst gap 48.5%

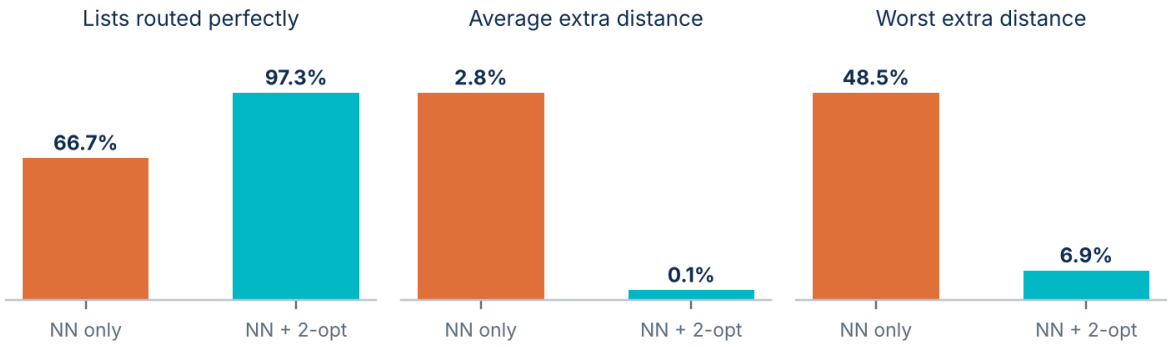
ID	Test	Result	Evidence
GC-08	Planning is fast enough for a cart screen (< 50 ms for 16 items)	**PASS**	full 16-item list: 2.71 ms average
GC-09	Demo list matches the numbers the site shows	**PASS**	default 6 items: aisle-by-aisle 216 m, GlideCart 99 m, 54% shorter, 1.6 min saved; stops: Bananas › Milk › Eggs › Pasta › Coffee › Sliced turkey
GC-10	Untouched price file is accepted	**PASS**	digest 5ebf636355c23f4d…
GC-11	Site tamper case (Apples 2.49 to 0.49) is blocked	**PASS**	mismatch detected
GC-12	Any one-character change to the file is blocked	**PASS**	126 of 126 single-character edits caught

Run 2026-10-29 with Node v22.23.4. Test code: tests/test_glidecart.js.

Route quality



Average walking saved against the aisle-by-aisle path, 1,000 random lists. Short lists save the most because the cart skips whole aisles.



Against the perfect route (brute force, 150 lists of 3-7 items): the 2-opt step lifts perfect routes from 67% to 97% and cuts the worst case from 48% extra to 7%.

Findings

#	Severity	Finding	Recommended fix
F1	**Medium**	The price-file check compares SHA-256 digests. That proves the file matches the digest, but anyone able to swap the file could also swap a digest sent the same way. It is an integrity check, not a signature.	Supplier signs the file with a private key (ML-DSA, NIST FIPS 204, or ECDSA P-256). The cart verifies with a pinned public key. Add a test that a file signed with the wrong key is rejected.
F2	Low	On the store maps the route line covers the "Checkout" label and the numbers for aisles 1 and 2.	Draw labels after the route, with a white outline.
F3	Low	Checkout shows "You walked 99 m", but this is the planned route length, not distance actually walked.	Rename to "Planned route", or measure with wheel sensors in the hardware build.
F4	Info	Lab 2 algorithm recommender scores 100% on synthetic data, a sign of labels a model can copy exactly.	Retrain on real or noisier data and report a confusion matrix.

Launch test plan for the AI, voice and ML features

These tests are designed, not yet run, because the features are designed, not yet built. Each has a pass mark agreed before testing starts.

ID	What is tested	How	Pass when
AI-01	Speech recognition accuracy	Recorded set of 1,000 commands with store background noise at about 70 dB	Word error rate 10% or less
AI-02	Intent understanding	500 labelled phrases, including slang and two-item commands	95%+ correct; every confusion reviewed
AI-03	Voice stop	Cart moving in Follow mode, "stop" spoken from 1-2 m	Cart halts within 300 ms of the end of the word; button still works with mic off
AI-04	Audio never leaves the cart	Network capture and storage scan across 100 voice sessions	Zero audio bytes sent or stored
AI-05	Sensor lights tell the truth	Hardware-in-the-loop: log mic and camera state against light state	100% match, including startup and crashes
AI-06	Product recognition	Held-out photos of every product, in bags and partly covered	Top-1 97%+; wrong confirmation under 1%; unsure cases fall back to tap
AI-07	Price anomaly detector	Inject 200 bad price changes into a year of clean history	95%+ caught; 2% or fewer false alarms
AI-08	Fairness and access	Speakers grouped by accent, age and speech differences	Error rate gap between groups 3 points or less; all features reachable by touch
AI-09	No made-up answers	200 price and location questions, 50 about items the store does not carry	100% match the price file and map; all 50 answered "not carried"
AI-10	Real-store value	Four-week pilot from the For stores page	Shorter walks and pick times than the store's normal path; shopper trust rated 4/5 or higher

Accessibility and security checks for every release

- Keyboard-only walk-through of all five pages and the cart screen; screen reader check (VoiceOver, NVDA) of maps and live results.
- Colour contrast of route colours and status text to WCAG 2.2 AA; reduced-motion setting stops the route animation.
- Signature tests from F1: valid file accepted, edited file blocked, wrong key blocked, expired key blocked.

07 · Next steps

Roadmap

Phase	Work	Done when
1. Fix findings	Real signatures (F1), map labels (F2), checkout wording (F3)	Tests GC-10 to GC-12 rewritten as signature tests and passing
2. Voice and assistant	On-device speech model, intent classifier, Glide assistant on the cart screen	AI-01 to AI-05 and AI-09 pass
3. Learning features	Product recognition, price anomaly alerts, trip-time model	AI-06 and AI-07 pass on real store data
4. Store pilot	Four-week, one-store pilot with opt-in shoppers and pickers	AI-08 and AI-10 pass

What this project shows

End-to-end product thinking: a retail problem from real operations experience, a UX built around privacy, working algorithms with measured results, a security review that found a real gap, and AI features specified with pass marks before any code is written.